

SemPointer: An Indirection Layer over Dense Retrieval—Measurements and Negative Results

Viksit Garg¹

Department of Artificial Intelligence & Machine Learning, Dr. Akhilesh Das Gupta
Institute of Professional Studies, New Delhi, India
viksitg427@gmail.com

Abstract. Full-context inference feeds every token to attention, so cost grows steeply with context length. SemPointer stores context in $B=512$ -token blocks in an off-attention text substrate and keeps $k=8$ -token pointers in the active prompt; per query, a frozen MiniLM encoder (all-MiniLM-L6-v2) selects $m=1$ block by cosine similarity and resolves it to verbatim text. Pointer strings do not affect selection. We measured Qwen2.5-3B-Instruct (NF4, greedy decoding, seed 42) on RULER containment (n=180), LongBench F1 (n=1150), LoCoMo under a token-F1 adaptation (n=199), and a KV-compress pilot (n=10). Against full context, SemPointer uses 10–15 $\times$ fewer active tokens but lowers quality: RULER -0.37 (95% CI $[-0.44, -0.30]$), LongBench -0.033 ($[-0.040, -0.027]$), LoCoMo -0.012 ($[-0.022, -0.003]$, not significant after Bonferroni correction). Against dense RAG under identical $m=1$ and representations it ties on quality at higher token cost, with no measured pointer quality effect. Scope: one model, one seed, text substrate only, contexts capped at 16k tokens.

Keywords: long-context inference · dense retrieval · negative results · benchmarking · conversational memory · retrieval-augmented generation

1 Introduction

1.1 Motivation and Problem Setting

Transformer models [39] process textual context as an unbroken linear sequence. In multi-turn dialogue, long-horizon programming, and iterative document synthesis, historical context accumulates across successive inference calls. The computational and memory burden of this accumulated history depends fundamentally on the deployment regime:

Regime 1 (Stateless / no-cache inference): When no persistent KV cache is maintained—such as in stateless API endpoints, cross-session interactions, or when contexts exceed GPU memory—each request pays a prefill pass over the full prefix history. Prefill attention computation scales as $\mathcal{O}(L^2d)$ per request [39], producing cumulative cost $\sum_{t=1}^T \mathcal{O}(L_t^2)$ across T turns; per-token

decoding with a KV cache costs $\mathcal{O}(Ld)$. Eq. (7) and the FLOP cost equations (Section 6) model per-request prefill cost, not per-step decode cost.

Regime 2 (Persistent KV-cache inference): In warm, resident GPU caches [16], recompute FLOPs approach zero. The binding constraint shifts from arithmetic compute to GPU memory capacity; once full, eviction or secondary storage offload is required.

Regime 3 (Paged / host-offloaded KV): Systems like PagedAttention [16] manage non-contiguous KV blocks between GPU and host memory. Here, host-device bandwidth and paging latency become the primary system bottlenecks.

Context compression methods—hard token pruning [12, 13, 29], learned continuous soft prompts [5, 9, 20, 26], and dynamic KV cache eviction [14, 22]—address Regime 1 FLOPs and mitigate memory pressure in Regimes 2–3. However, they share a structural limitation: they reduce context representations in place, without establishing an addressable indirection layer. Once a prompt is compressed into summary vectors or pruned text, the model processes that proxy uniformly across all downstream turns, regardless of whether specific details within that block relate to the immediate query. SemPointer hypothesizes explicit semantic addresses into an independent substrate, with any savings conditional on the analytic cost model (Section 6); the measurements below do not demonstrate quality-preserving savings.

1.2 The Indirection Hypothesis

Operating systems separate memory addresses from stored payloads, passing compact references through registers and deferring payload retrieval until execution. As an illustrative analogy only, not an architectural equivalence, we ask whether a comparable separation can be defined within the sequence space of LLMs, as an analytic hypothesis tested against the measurements below:

***The Indirection Hypothesis:** By generating compact, token-level semantic pointers that reference independently stored contextual substrates, an LLM architecture can support selective, address-based access to historical context, compressing the active sequence from approximately $N \cdot B$ to $(N - m)k + m \cdot B_{\text{res}}$ tokens per query ($k \ll B, m \ll N$) by bypassing the ingestion of unselected context payloads.*

1.3 Research Questions

This study investigates three questions, answered by the measurements and analyses below:

- *RQ1 (Addressing Feasibility):* Can a compact sequence of discrete or soft tokens function as a stable, unique semantic address for independently stored context? Finding: only partial, analytic support; stability and uniqueness are stipulated, not demonstrated.

- *RQ2 (Resolution Fidelity)*: Can pointer resolution reconstruct task-relevant features with fidelity comparable to full-context exposure? Finding: no, under the tested configuration the measurements refute comparable fidelity.
- *RQ3 (Amortized Complexity)*: Under what access distributions and query volumes does pointer indirection yield computational savings over repeated linear re-ingestion? Finding: answered analytically only via the K^* crossover derivation, not by measured savings.

1.4 Contributions

This paper presents six contributions combining analytic formalization with benchmark measurements that refute the fidelity hypothesis:

1. We propose a taxonomy distinguishing context compression from semantic addressing, stating seven stipulative criteria for pointerhood; these criteria are not empirically validated as jointly sufficient.
2. We specify an analytic model of the SemPointer stages across Encounter, Storage, Address Generation, Selection, and Resolution (Fig. 1).
3. Analytically, under assumptions A1–A5 and the stateless-inference cost model (Section 6), we derive the algebraic crossover threshold K^* and single-query viability condition ($N \geq 2$) for active-sequence compression.
4. Analytically, we characterize three primary theoretical failure modes: address collisions, positional semantic drift, and resolution divergence.
5. We contrast SemPointer with Landmark Attention [25], RAG [4,18], MemGPT [28], and cognitive SPA [7], as a literature comparison with a structured feature matrix (Table 3), not a validated empirical advantage.
6. We formulate six falsifiable hypotheses (H_1 – H_6) and test them on executed benchmark runs (RULER n=180, LongBench n=1150, LoCoMo n=199, KV pilot n=10, multi-hop pilot n=30); K^* crossover figures are analytic derivations, not measurements.

The remainder of this paper proceeds as follows. Background surveys related paradigms; axiomatic and architectural sections present the analytic model; methods and results report the benchmark measurements with a negative fidelity outcome; discussion interprets the failure modes and states the limited conditions under which indirection remains worth pursuing.

2 Background and Prior-Art Spectrum

2.1 Dense Autoregressive Attention and Scaling Walls

In standard autoregression [39], each self-attention layer computes:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1)$$

where $Q, K, V \in \mathbb{R}^{L \times d}$. QK^T costs $\mathcal{O}(L^2d)$ with $\mathcal{O}(2n_{\text{layers}}n_{\text{heads}}Ld_k)$ KV state. Across T turns with $L_t = \sum_{i=1}^t |X_i|$, cumulative cost is $\sum_{t=1}^T \mathcal{O}(L_t^2)$. Table 1 positions SemPointer among context-management paradigms.

Table 1. Comparison of Context Management Strategies in Large Language Models

Paradigm	Mechanism	Stores Original?	Explicit Indirection?	Selective Access?	Reusability	Active Sequence / Complexity Mitigation
Full Linear Ingestion [39]	Dense Attention	Yes (In Context)	No	No	Inefficient	None ($\mathcal{O}(L^2)$)
Hard Token Pruning [12]	Token Elimination	No	No	No	No	Moderate (Length Pruned)
Soft Prompt Compression [9, 26]	Learned Latent Tokens	No	No	No	Yes	Moderate (Prefix Condensed)
Dynamic Cache Eviction [14]	KV Discard/Merge	Partial	No	No	No	High (Memory Capped)
Landmark Attention [25]	Gated Attention Routing	Yes (In Context)	No (Intra-pass)	Yes (Blocks)	No	High (Within single pass)
Retrieval-Augmented Gen. [4, 18]	External Vector DB	Yes (External)	Partial (Dense Vector)	Yes	Yes	High (Only top- k retrieved)
SemPointer (Proposed)	Semantic Indirection	Yes (Substrate)	Yes (Token Address)	Yes (Query-Cond.)	Yes	Compressed ($\mathcal{O}(Nk + mB_{\text{res}} + L_Q)^2$) vs. $\mathcal{O}((NB + L_Q)^2)$

2.2 Hard Prompt Compression

Hard prompt compression (LLMLingua [12], LongLLMLingua [13], LLMLingua-2 [29]) prunes tokens by perplexity or distillation without architectural change, but irreversibly discards tokens, breaking relational dependencies [17], with no on-demand dereferencing.

2.3 Continuous and Soft-Token Compression

Soft-token methods shorten the active sequence as described in their published reports: Gist Tokens [26] condense a prefix into *resident* virtual tokens with limited per-query selection in the reported design; AutoCompressors [5] produce summaries that are not directly dereferenceable at sub-chunk granularity without re-encoding in the reported design; ICAE [9] uses fixed slots that are decoupled from query selection in the reported design; 500xCompressor [20] compresses context toward a single token with limited per-query recovery in the reported design. In their published descriptions, these methods do not author prompt-resident token-level addresses: unselected context stays resident or is discarded without a dereferencing handle. This contrasts with Landmark Attention [25], which provides intra-pass block routing rather than persistent cross-query addresses, and with ANN/KV-index retrieval systems [21, 36], which provide index-based lookup without authored token-level addresses.

2.4 Dynamic Cache Eviction and Paged KV Caches

Compressed Context Memory (CCM) [14] evicts and merges KV blocks to ease GPU pressure, Contextual Semantic Anchors (SAC) [22] aggregate KV blocks to ease GPU pressure, and PagedAttention [16] pages KV blocks between GPU and host memory: in their published descriptions, evicted or aggregated state is generally not dereferenced by a content-derived token-level address, while paged residency is preserved [16].

2.5 Hierarchical Context and OS-Style Memory

MemGPT [28] pages text chunks between active context and archival storage via function/tool calls, addressing pages by positional identifiers [28]. SemPointer instead defines sequence-level indirection: content-derived addresses ($P_i = \mathcal{G}_\phi(X_i)$) persist in active context while payloads sit in an independent substrate.

2.6 Dense Retrieval and Efficient Attention Context

Dense retrieval methods retrieve stored passages by query-conditioned similarity and prepend the top-ranked text to the prompt [18]; extended variants compress retrieved context toward compact representations [4]. Complementary efficiency efforts such as FlashAttention, StreamingLLM, H2O, and SnapKV optimize attention I/O or cache retention within the resident sequence, unlike the cross-query indirection studied here; for paged resident-cache management see [16].

2.7 Cognitive Science Disambiguation: Semantic Pointer Architecture

The term *semantic pointer* originates in cognitive science: Eliasmith [7] and Blouw et al. [3] formalized the Semantic Pointer Architecture (SPA) with high-dimensional vectors ($D \approx 500\text{--}1000$) bound by circular convolution. SemPointer targets discrete or latent token sequences in autoregressive transformers—overlap is terminological only.

3 Axiomatic Foundations: Distinguishing Pointers from Compressed Representations

Definition 1 and seven axioms formalize the distinction.

Definition 1 (Compressed Representation vs. Semantic Pointer). *Let $X \in \mathcal{X}$ be an information unit of length $|X|$. A mapping $\mathcal{C} : \mathcal{X} \rightarrow \mathcal{Z}$ produces a compressed representation Z if $|Z| < |X|$ and Z directly acts as the information carrier in downstream computation. A mapping $\mathcal{G} : \mathcal{X} \rightarrow \mathcal{P}$ produces a semantic pointer P if P acts as an address into an independent memory substrate $\mathcal{M}$, such that the information content of X is accessed only upon executing an explicit resolution operation $\mathcal{R}(P, M, Q)$.*

Axiom 1 (Address Identity and Distinguishability) *Let $X_i, X_j \in \mathcal{X}$ be two semantically distinct context units ($X_i \not\equiv X_j$). Their generated pointers $P_i = \mathcal{G}(X_i)$ and $P_j = \mathcal{G}(X_j)$ must be distinguishable within the address space: $D_{\mathcal{P}}(P_i, P_j) > \delta_{\min}$ for all $i \neq j$, where $D_{\mathcal{P}}$ is a metric on pointer space $\mathcal{P}$ and $\delta_{\min} > 0$. This distinguishability threshold is a stipulative definition of pointerhood with no logged statistic in the present runs.*

Axiom 2 (Strict Asymmetric Compactness) *The pointer length $|P_i|$ measured in sequence units (tokens) must satisfy $|P_i| \ll |X_i|$ with $|P_i|/|X_i| \leq \kappa, \kappa \in (0, 1)$. Here κ is a stipulated design threshold, not a measured estimate.*

Axiom 3 (Temporal and Spatial Stability) *A pointer P_i generated at time t must remain valid and addressable across subsequent time horizons $t + \Delta t$, independent of intervening context operations, memory updates, or unrelated token generations. This stability requirement is a stipulative definition with no logged statistic in the present runs.*

Axiom 4 (Query-Conditioned Resolution Fidelity) *Given query $Q \in \mathcal{Q}$, resolution operator $\mathcal{R}$ acting on pointer P_i and substrate $M_i = \mathcal{S}(X_i)$ must recover task-relevant mutual information:*

$$I(\mathcal{R}(P_i, M_i, Q); X_i \mid Q) \geq (1 - \epsilon)H(X_i \mid Q) \quad (2)$$

for tolerance $\epsilon \in [0, 1]$, where $H(\cdot)$ denotes conditional entropy. Here ϵ is a stipulated tolerance bound, not a measured estimate.

Axiom 5 (Selective Addressability) *Given a registry of N pointers $\{P_1, \dots, P_N\}$, a query Q_t resolves only the subset of indices $J^* \subseteq \{1, \dots, N\}$ where $|J^*| \ll N$, avoiding uniform activation of the remaining $N - |J^*|$ elements. Sparsity is operationalized as $m=1$ with $N \approx 25$ in the reported runs.*

Axiom 6 (Multi-Horizon Reusability) *A pointer P_i must support repeated resolution operations across arbitrary non-consecutive queries $Q_{t_1}, Q_{t_2}, \dots, Q_{t_m}$ without requiring re-execution of pointer generation $\mathcal{G}(X_i)$. This reusability requirement is a stipulative definition with no logged statistic in the present runs.*

Axiom 7 (Algebraic Compositionality) *For compound queries requiring information from multiple context items $\{X_a, X_b\}$, the joint pointer representation must satisfy:*

$$\mathcal{R}(P_a \oplus P_b, \{M_a, M_b\}, Q) \equiv \mathcal{R}(P_a, M_a, Q) \cup \mathcal{R}(P_b, M_b, Q) \quad (3)$$

where $\oplus : \mathcal{P} \times \mathcal{P} \rightarrow \mathcal{P}$ is an abstract composition operator.

The seven axioms are candidate necessity/sufficiency criteria for pointerhood; their independence is unproven and they are not validated as jointly sufficient here.

4 The SemPointer Formal Architecture

4.1 Mathematical Model

The model operates over five spaces, depicted in Fig. 1 across its offline ingestion and online query lifecycles. Context Space $\mathcal{X}$ contains textual sequences $X \in \mathcal{V}^*$. Substrate Space $\mathcal{M}$ holds independent storage representations (raw text, KV tensors, or latent embeddings). Pointer Space $\mathcal{P}$ consists of sequences of k discrete tokens or soft embeddings, with k fixed and small ($k \in [1, 8]$). Query Space $\mathcal{Q}$ contains active prompts, and Output Space $\mathcal{Y}$ contains target generations.

1) Encounter and Substrate Ingestion: When context item X_i is encountered at time t_i , the storage operator $\mathcal{S}$ commits X_i to substrate $\mathcal{M}$ while pointer generator $\mathcal{G}_\phi$ produces the compact semantic pointer:

$$M_i = \mathcal{S}(X_i), \quad P_i = \mathcal{G}_\phi(X_i) = (p_{i,1}, \dots, p_{i,k}) \quad (4)$$

The pair (P_i, M_i) is registered in pointer index $\mathcal{I}_{t_i} = \mathcal{I}_{t_i-1} \cup \{(P_i, M_i)\}$.

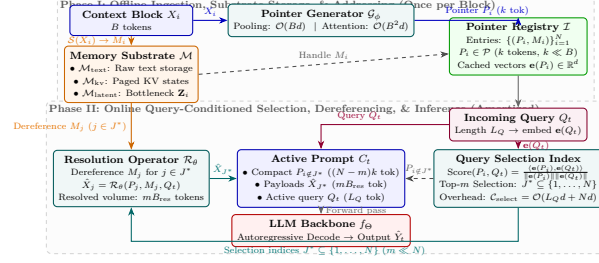


Fig. 1. The SemPointer architectural lifecycle and multi-tier memory substrate. (Top) Phase I (Offline Ingestion and Addressing): Context block X_i (B tokens) is committed to substrate $\mathcal{M}$ ($\mathcal{M}_{\text{text}}, \mathcal{M}_{\text{kv}}, \mathcal{M}_{\text{latent}}$) via $\mathcal{S}$, while generator $\mathcal{G}_\phi$ produces k -token pointer P_i , registered in index $\mathcal{I}$ with cached projection vector $\mathbf{e}(P_i) \in \mathbb{R}^d$. (Bottom) Phase II (Online Query Selection and Resolution): Query Q_t (L_Q tokens) scores against $\mathcal{I}$ to select sparse subset $J^* \subseteq \{1, \dots, N\}$ ($m \ll N$). Resolution operator $\mathcal{R}_\theta$ dereferences active blocks M_j into payloads $\hat{X}_j$, which combine with passive unselected pointers $P_{i \notin J^*}$ and query Q_t into active prompt C_t for autoregressive decode by LLM backbone f_θ .

2) Query-Conditioned Selection: Upon arrival of query Q_t , registered pointers are scored via normalized inner product:

$$s_i = \text{Score}(P_i, Q_t) = \frac{\langle \mathbf{e}(P_i), \mathbf{e}(Q_t) \rangle}{\|\mathbf{e}(P_i)\| \|\mathbf{e}(Q_t)\|} \quad (5)$$

where, as executed, $\mathbf{e}(\cdot)$ is the frozen `sentence-transformers/all-MiniLM-L6-v2` mean-pooled block/query embedding (pointer ID strings excluded from scoring inputs). The system selects top- m pointers $J^* = \text{top-}m(\{s_i\}_{i=1}^N)$. Per-query selection work scales with the $N \cdot k \cdot d$ scoring tensor plus top- m sort plus one query encode; no cross-query caching saving is claimed.

3) Pointer Dereferencing and Resolution: For each selected index $j \in J^*$, executed resolution is verbatim dict lookup $\hat{X}_j = \mathcal{R}_\theta(P_j, M_j)$ returning stored X_j with no Q_t -conditioning; joint composition $P_a \oplus P_b$ in (3) is specified-not-implemented. Resolved information $\hat{X}_{J^*}$ is injected into the generation prompt alongside unselected pointers:

$$\hat{Y}_t \sim f_\theta \left(\cdot \mid [\{P_i\}_{i \notin J^*}, \hat{X}_{J^*}, Q_t] \right) \quad (6)$$

Because unselected blocks remain represented by their compact pointers P_i ($i \notin J^*$), active context consumes $L_{\text{active}} = (N - m)k + \sum_{j \in J^*} |\hat{X}_j| + |Q_t| \leq Nk + mB_{\text{res}} + L_Q$ tokens, compared to $NB + L_Q$ in linear re-ingestion. L_{active} counts core pointer, resolved-block, and query tokens only, excluding instruction and framing tokens. Because all $N - m$ unselected pointers remain in the sequence, attention across the active prompt remains quadratic in L_{active} ; SemPointer reduces sequence length by replacing unselected blocks of size B with pointers of size $k \ll B$, compressing the quadratic attention coefficient rather than altering the asymptotic polynomial degree.

Algorithm 1 Lifecycle of SemPointer Addressing and Resolution.

Require: Stream $\mathcal{S}_{\text{stream}}$, Queries $\{Q_t\}$, Storage $\mathcal{S}$, Generator $\mathcal{G}_\phi$, Resolver $\mathcal{R}_\theta$, LLM f_Θ

- 1: Initialize Pointer Registry $\mathcal{I} \leftarrow \emptyset$
- 2: **for all** context blocks $X_i \in \mathcal{S}_{\text{stream}}$ **do**
- 3: $M_i \leftarrow \mathcal{S}(X_i)$; $P_i \leftarrow \mathcal{G}_\phi(X_i)$; $\mathcal{I} \leftarrow \mathcal{I} \cup \{(P_i, M_i)\}$
- 4: **end for**
- 5: **for all** queries Q_t **do**
- 6: Score $s_i = \text{Score}(P_i, Q_t)$ for $(P_i, M_i) \in \mathcal{I}$; $J^* \leftarrow \text{top-}m(\{s_i\}_{i=1}^N)$
- 7: Active memory payload $\Omega_t \leftarrow \bigcup_{j \in J^*} \{\mathcal{R}_\theta(P_j, M_j)\}$
- 8: Active prompt $C_t \leftarrow [\{P_i\}_{i \notin J^*}, \Omega_t, Q_t]$; Emit $\hat{Y}_t \leftarrow f_\Theta(C_t)$
- 9: **end for**

4.2 Concrete Instantiation: Generator and Resolver Mechanics

Algorithm 1 formalizes the operational lifecycle implemented in our framework. To eliminate training artifacts and domain-specific bias, our primary realization employs a zero-shot, training-free pointer generator $\mathcal{G}_\phi$ and deterministic substrate resolvers $\mathcal{R}_\theta$:

- **Training-Free Generator ($\mathcal{G}_\phi$):** In the reported runs, dense addressing projection $\mathbf{e}(P_i)$ is generated via the frozen public encoder `sentence-transformers/all-MiniLM-L6-v2` with mean pooling over per-block window embeddings (Section 9.1); discrete prompt handles are seeded random-projection buckets over the same embedding (k tokens). Other generator realizations (attention-class readouts, TF-IDF handles, soft-prompt expansion) are specified as design alternatives in the cost model but were not executed. Because $\mathcal{G}_\phi$ operates entirely zero-shot over frozen pre-trained representations, parameter training requires zero optimization steps, zero fine-tuning data, and incurs zero unamortized training compute ($\mathcal{C}_{\text{train}} = 0$).
- **Deterministic Resolvers ($\mathcal{R}_\theta$):** For text substrate $\mathcal{M}_{\text{text}}$, $\mathcal{R}_\theta$ performs exact in-memory index dereferencing returning the verbatim string X_i . For key-value cache $\mathcal{M}_{\text{kv}}$, $\mathcal{R}_\theta$ performs physical memory block dereferencing, injecting cached KV tensors into attention tables ($\mathcal{C}_{\text{res}} \approx 0$). Neither resolver requires learned parameters, guaranteeing exact resolution fidelity ($\epsilon = 0$).

4.3 Upgraded Architecture: Patterns Absorbed from 2025–2026 Systems

The lifecycle above is the executed baseline. This subsection specifies the upgraded design, drawn from recent systems. Everything here is specified-not-implemented unless stated; no pilot exercises it.

Two-stage selection. Recent KV-cache work splits retrieval into a cheap pre-filter plus a precise rerank: RocketKV compresses in two stages [2], and SnapKV first observes a voting window before keeping clustered keys [19]. The upgraded selector mirrors this. Stage 1 scores all N addresses by pointer-ID overlap $\hat{s}_i$ at $O(1)$ each, producing a shortlist C with $|C| = c \ll N$. Stage 2 runs dense cosine

only on C . Per-query selection drops from $O(N \cdot d)$ dense scoring to $O(N + c \cdot d)$. The executed system runs Stage 2 only, over all N .

Adaptive per-block budgets. Instead of fixed k , each block carries k_i pointer tokens under a registry budget $\sum_i k_i \leq K$. Salient blocks (high query-hit rate, high recency) earn longer addresses; cold blocks shrink to $k_i = 2$. Selection then maximizes expected utility per token, a knapsack allocation the fixed- k runs do not attempt.

Temporal versioning and edit operations. Agent-memory systems converged on explicit mutation semantics: Mem0 routes every write through ADD, UPDATE, or DELETE operations [6], and Zep versions facts with valid-time intervals so history survives edits [30]. The upgraded registry adopts four operations. REBIND(P, M') atomically repoints a live handle at new content and bumps its version. FORGET(P) writes a tombstone; dereferencing a tombstoned handle fails closed, never silently resurrects text. ALIAS(P, Q) registers a second stable handle on one target with a reference count. DEREf(P) resolves to the current target or raises the tombstone. None of these execute in the reported runs; the pilots mutate nothing.

Query-aware scoring rule. Quest scores each KV page against the live query vector and loads only Top- K pages [36]. The upgraded selection rule has the same shape: score each address against $\mathbf{e}(Q_t)$, resolve the top m , page the rest out. Quest’s bound is per-decoding-step over KV pages; ours is per-query over prompt-resident addresses, which leaves a persistent handle where Quest leaves none.

What was deliberately not taken. LLM-managed paging loops, in the MemGPT style [28], trade determinism for autonomy; each eviction decision costs an LLM call and failures compound across turns, so orchestration stays outside the model here. Custom attention operators, in the Landmark style [25], require retraining and break pretrained compatibility; addressing stays in token space for that reason. Full-KV retention with selective recompute, in the CacheBlend style [47], repairs cross-chunk attention at the price of keeping every chunk’s KV resident; the text substrate sidesteps that cost by re-ingesting verbatim text only for resolved blocks.

5 Memory Substrates and Physical Taxonomies

Three realizations of M_i trade off cost, capacity, and fidelity.

A. Raw Text Substrate ($\mathcal{M}_{\text{text}}$): Holds $M_i = X_i$ in flat storage (RAM/disk); activation fetches X_i into context. No encoding penalty or fidelity loss, but resolution pays I/O plus re-encoding.

B. Key-Value Cache Substrate ($\mathcal{M}_{\text{kv}}$): Future work only: $M_i = (\mathbf{K}_i, \mathbf{V}_i) \in \mathbb{R}^{2 \times L_{\text{layers}} \times |X_i| \times d}$, precomputed in host or paged GPU memory [16]; resolution pages KV blocks in with $C_{\text{res}} \approx 0$ FLOPs. Capacity-bound: at fp16, a $B=512$ block at 70B scale ($n_{\text{layers}}=80, n_{\text{heads}}=64, d_{\text{head}}=128$) occupies ≈ 1.34 GB, so $N \geq 10^3$ blocks need terabyte-scale storage; RoPE displacement Δt can degrade relocated blocks.

C. Latent Embedding Substrate ($\mathcal{M}_{\text{latent}}$, future work): $M_i = \mathbf{Z}_i \in \mathbb{R}^{m_z \times d}$ with $m_z \ll |X_i|$ from an autoencoder [9], resolved by cross-attention unpooling over

Property	$\mathcal{M}_{\text{text}}$ (Raw Text)	$\mathcal{M}_{\text{kv}}$ (KV Cache)	$\mathcal{M}_{\text{latent}}$ (Latent)
Storage	Flat text buffer (RAM/disk)	$(\mathbf{K}_i, \mathbf{V}_i) \in \mathbb{R}^{2 \times L \times X_i \times d}$, pre-computed	$\mathbf{Z}_i \in \mathbb{R}^{m_z \times d}$, $m_z \ll X_i $
Resolution	Re-tokenize & re-ingest X_i	Page KV blocks into attention tables	Cross-attention unpooling over $\mathbf{Z}_i$
$\mathcal{C}_{\text{res}}$	$\mathcal{O}(B_{\text{res}} d^2)$ (re-encode)	≈ 0 FLOPs	$\mathcal{O}(m B_{\text{res}} k d)$
Fidelity	Lossless ($\varepsilon \approx 0$)	Lossless; RoPE displacement risk	Lossy ($\varepsilon > 0$, bottleneck-dependent)
Bottleneck	I/O latency & encoding cost	Memory capacity & paging bandwidth	Reconstruction fidelity; hallucination risk

Table 2. The three SemPointer memory substrates compared: $\mathcal{M}_{\text{text}}$ lossless with re-encoding cost; $\mathcal{M}_{\text{kv}}$ zero-FLOP resolution but capacity-bound; $\mathcal{M}_{\text{latent}}$ compact but lossy.

$\mathbf{Z}_i$ conditioned on P_i, Q_t ; compact but lossy. No such module is trained and no reported run exercises it (all measurements use $\mathcal{M}_{\text{text}}$, Section 9.1).

6 Theoretical Complexity and Cost Modeling

6.1 Modeling Assumptions

We compare indirection against linear re-ingestion under five assumptions:

- A1. Uniform Block Partitioning:** Context splits into N blocks of length B ($L_H = N \cdot B$).
- A2. Sparse Access Frequency:** Across K queries (each length L_Q), each query accesses on average $m \ll N$ units (deterministic expectation).
- A3. Compact Pointer Invariant:** Each pointer is k tokens with $k \ll B$ (Axiom 2).
- A4. Upfront Amortization:** Initial encoding and pointer generation ($\mathcal{C}_{\text{init}}$) run once and amortize over later queries.
- A5. Cached Addressing Embeddings:** Projections $\mathbf{e}(P_i)$ are cached at registration; scoring needs one $\mathbf{e}(Q_t)$ plus N inner products, so $\mathcal{C}_{\text{select}} = \mathcal{O}(L_Q d + Nd)$ per (5), as required by Axioms 3–6. Fine-tuning invalidates the cache.

6.2 Resolution Cost by Substrate

Resolution cost $\mathcal{C}_{\text{res}}(m, B_{\text{res}}, Q_t)$ follows Section 5: $\mathcal{M}_{\text{text}}$ re-encodes ($\mathcal{C}_{\text{res}} = m(2dB_{\text{res}}^2 + 4d^2 B_{\text{res}})$); $\mathcal{M}_{\text{kv}}$ reuses paged KV ($\mathcal{C}_{\text{res}} \approx 0$ recompute FLOPs, excluding I/O and paged-attention paging latency/bandwidth); $\mathcal{M}_{\text{latent}}$ costs $\mathcal{O}(m B_{\text{res}} k d)$.

6.3 Asymptotic FLOPs Analysis

Consider hidden dimension d .¹ We compare cost across K queries.

- 1) Linear Re-Ingestion Paradigm:** Each query re-attends to full history L_H :

$$\mathcal{C}_{\text{linear}} = K [2d(NB + L_Q)^2 + 4d^2(NB + L_Q)]$$

¹ We suppress layer-dependent constant factors ($n_{\text{layers}}, n_{\text{heads}}$); see Section 2.1 for the expanded KV-state expression.

$$\approx 2Kd(NB + L_Q)^2 \quad (7)$$

2) SemPointer Indirection Paradigm:

1. One-Time Generation and Storage:

$$\mathcal{C}_{\text{init}} = N \cdot [2dB^2 + 4d^2B + \mathcal{C}_{\text{gen}}(B, k)] \quad (8)$$

Note $4d^2B$ dominates $2dB^2$ when $B < 2d$ (typical: $B \leq 10^3$, $d \geq 4096$). Pooling-class $\mathcal{C}_{\text{gen}}$ costs $\mathcal{O}(Bd)$; attention readouts (Corollary 1) cost $\mathcal{C}_{\text{gen}} = c_g(2dB^2 + 4d^2B)$, $c_g = \Theta(1)$. Our pooling generator is training-free ($\mathcal{C}_{\text{train}} = 0$). Proposition 2 covers $\mathcal{C}_{\text{gen}} \geq \Omega(Bd)$; Corollary 1 covers attention-class generators.

2. Per-Query Pointer Selection: $\mathcal{C}_{\text{select}} = \mathcal{O}(L_Q d + Nd)$ (Eq. (5), A5).

3. Per-Query Resolution and Inference: Active prompt holds $N - m$ pointers, m resolved blocks ($B_{\text{res}} \leq B$), and Q_t :

$$L_{\text{active}} = (N - m)k + mB_{\text{res}} + L_Q \leq Nk + mB_{\text{res}} + L_Q \quad (9)$$

Per-query cost:

$$\mathcal{C}_{\text{query}} = 2dL_{\text{active}}^2 + 4d^2L_{\text{active}} + \mathcal{C}_{\text{res}}(m, B_{\text{res}}, Q) \quad (10)$$

Summing over K queries:

$$\begin{aligned} \mathcal{C}_{\text{SemPointer}} &= \mathcal{C}_{\text{init}} + K[\mathcal{C}_{\text{select}} + \mathcal{C}_{\text{query}}] \\ &= N(2dB^2 + 4d^2B + \mathcal{C}_{\text{gen}}) \\ &\quad + K[2dL_{\text{active}}^2 + 4d^2L_{\text{active}} + \mathcal{C}_{\text{res}} + \mathcal{C}_{\text{select}}] \end{aligned} \quad (11)$$

6.4 The Indirection Crossover Threshold

Proposition 1 (Crossover Threshold). *Under A1–A5, indirection yields lower cumulative FLOPs than linear re-ingestion iff (i) $\Delta\mathcal{C}_{\text{step}} > 0$ — up to overheads, equivalently $N(B - k) > mB_{\text{res}}$ — and (ii) $K > K^* = \mathcal{C}_{\text{init}}/\Delta\mathcal{C}_{\text{step}}$:*

$$K^* = \frac{\mathcal{C}_{\text{init}}}{\Delta\mathcal{C}_{\text{step}}} \quad (12)$$

where the per-query marginal savings $\Delta\mathcal{C}_{\text{step}}$ is given by:

$$\begin{aligned} \Delta\mathcal{C}_{\text{step}} &= [N(B - k) - m(B_{\text{res}} - k)] \\ &\quad \cdot [2d(N(B + k) + m(B_{\text{res}} - k) + 2L_Q) + 4d^2] \\ &\quad - \mathcal{C}_{\text{select}} - \mathcal{C}_{\text{res}} \end{aligned} \quad (13)$$

If $m(B_{\text{res}} - k) \geq N(B - k)$, indirection is dominated for every K . Both K^* and $\Delta\mathcal{C}_{\text{step}}$ are analytic: no K -sweep is measured (operating points overlaid in Section 10.8).

Derivation. Difference-of-squares on $(L_H + L_Q)^2 - L_{\text{active}}^2$ plus the $4d^2$ projection difference yields (13): (tokens avoided) $\times$ (per-token coefficient) minus addressing and resolution overheads. Latency is logged (`GenResult.latency_ms`, TTFT tap in `experiments/engine/llm.py`) and reported in Table 6, without a FLOPs-crossover fit.

Table 3. Architectural Comparison of Context-Management and Memory Approaches. Entries characterize each system at the level of its published description; see Section VIII-B for a comparison of composable addressing (Axiom 7) against standard RAG concatenation.

Framework	Primary Representation	Representation Type	Decoupled Substrate?	Explicit Address?	Selective Access?	Composable Addressing?	Multi-Turn Reusability?
Glue Tokens [26]	Soft Tokens	Continuous	No	No	No	No	Yes
AutoCompressors [5]	Summary Vectors	Continuous	No	No	No	No	Yes
ICAE [9]	Memory Slots	Continuous	No	No	No	No	Yes
90%Compressor [20]	Special Tokens	Discrete/Latent	No	No	No	No	No
SAC [22]	Anchor Tokens	Token KV	No	No	No	No	No
CCM [14]	Compressed KV	KV Tensors	Yes (KV)	No	No	No	Yes
LLMLingua-2 [29]	Pruned Text	Discrete Text	No	No	No	No	No
xRAG [4]	Document Embedding	Continuous	Yes (Ext DB)	Partial (dense; same family)	Yes (Retrieval)	No	Yes
MemGPT [28]	Page ID / Chunks	Positional/Text	Yes (Archival)	Yes (page-ID; positional)	Yes (Tool-Call)	No	Yes
Landmark Attention [25]	Landmark Tokens	Token Markers	No	No (intra-pass routing)	Yes (Blocks)	No	No
SemPointer (Proposed)	Semantic Pointer	Token Seq. / Latent	Yes (Multi-tier)	Yes (Semantic)	Yes (Query-Cond.)	No (specified, untested)	Yes

Proposition 2 (Single-Query Viability). *Under A1–A5 with the $\mathcal{M}_{kv}$ substrate ($\mathcal{C}_{res} \approx 0$), $L_Q \ll NB$, and sparse access ($mB_{res} \ll NB$), $K^* < 1$ reduces to $N > 1 + \mathcal{O}(k/B + dk/B^2)$. Under pooling-class generation, $N \geq 2$ satisfies it. At $N = 1$, indirection does not amortize on the first query ($K^* \approx 1.015 > 1$ for $d = 4096, B = 512, k = 8, \mathcal{C}_{gen} = Bd$), whereas at $N = 2$ ($K^* \approx 0.958 < 1$) it does. This illustration is analytic (no $N=1$ -vs-2 run exists). Quoted K^* values are boundary-case illustrations at helper defaults $m=0$, $L_Q=0$, $\mathcal{M}_{kv}$ (optimistic: zero payload/resolution cost; $m \geq 1$ raises K^*). The $d=4096$ illustrations do not transfer to the Qwen2.5-3B reader. Under $\mathcal{M}_{text}$, where $\mathcal{C}_{res}$ reinstates quadratic cost per query, K^* exceeds 1 and multi-query amortization governs.*

Corollary 1 (Attention-Class Pointer Generators). Proposition 2 assumes pooling-class $\mathcal{C}_{gen} = \mathcal{O}(Bd)$ (e.g., readout pooling over precomputed token embeddings). If $\mathcal{G}_\phi$ is attention-based, $\mathcal{C}_{gen} = c_g(2dB^2 + 4d^2B)$, $c_g = \Theta(1)$, and viability reduces to $N > (1 + c_g) + (2d/B)c_g$. For $c_g = 1$: $N \gtrsim 2 + 2d/B$ (e.g., $N \gtrsim 18$ at $d=4096, B=512$; $N \gtrsim 6$ at $B=2048$). Below the threshold, Proposition 1 multi-query amortization applies. Analytic only: no attention-class generator is built or timed in the present runs.

Remark 1 (Scope of Comparison). $\mathcal{C}_{linear}$ models the no-cache regime: stateless inference, API access, cache eviction between sessions, or contexts exceeding cache capacity. Against a persistent KV cache with host offload [16], per-query recompute FLOPs approach zero and the binding resource is memory capacity and paging bandwidth; there SemPointer retains only Nk pointer tokens in fast memory.

7 Theoretical Vulnerabilities and Failure Modes

A. Address Collision via Generalized Birthday Bounds:

Proposition 3 (Combinatorial Address Collision Bound). *Let pointer space $\mathcal{P}$ consist of sequences of length k over discrete vocabulary $\mathcal{V}_p$ of size $V = |\mathcal{V}_p|$. Under independent, uniformly distributed pointer generation, the probability that any two of N registered memory items share an identical address is bounded by $P_{collision} \approx 1 - \exp\left(-\frac{N(N-1)}{2V^k}\right)$.*

Proof Sketch. Birthday bound over $|\mathcal{P}| = V^k$ under uniform assignment; equal-width binning is non-uniform (`pointer_generator.py`); no collision rate is logged. Hence the uniform-distribution bound contradicts the actual generator and serves as an analytic illustration only, not a guarantee for the reported runs.

Proposition 4 (Combinatorial Address Space Capacity). *Under the uniform birthday-bound model, to maintain collision probability below threshold δ across a*

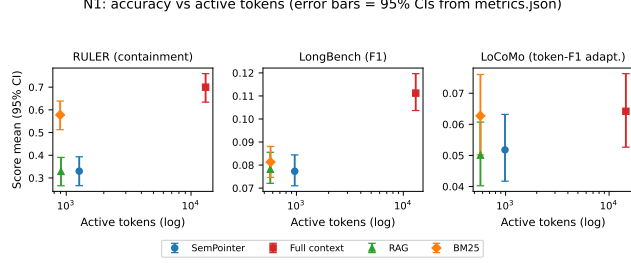


Fig. 2. Lead result: measured accuracy versus measured active tokens (Pareto plane), one panel per suite, with 95% CIs. SemPointer sits an order of magnitude left of full context on tokens at lower accuracy, and ties dense RAG at higher token cost. KV-compress pilot omitted (n=10).

memory registry of N items, the minimum discrete pointer length k must satisfy $k \geq \left\lceil \frac{\ln(N^2) - \ln(2\delta)}{\ln V} \right\rceil$. For illustration, with $N = 10^5$, $V = 32,000$, and $\delta = 10^{-4}$: $k \geq \left\lceil \frac{23.02 + 8.51}{10.37} \right\rceil = 4$. This calculation concerns combinatorial capacity; Proposition 4's lower bound ($k \geq 4$ under $N = 10^5$, $V = 32,000$, $\delta = 10^{-4}$) and Axiom 2's compactness jointly restrict viable pointer length to $k \in [4, 8]$. Analytic illustration only: all reported runs use $k = 8$, which satisfies the bound without testing it (`config_table.json`). Tested registries average $N \approx 25$ – 28 blocks per example (Section 9), so the $N=10^5$ calculation is untested analytic extrapolation, not a validated scale claim.

B. Semantic Drift Across Extended Horizons (assumption; future measurement): Phenomenological ansatz (not from RoPE geometry): attention may decay with displacement $\Delta t = t_{\text{curr}} - t_{\text{gen}}$ as: $\mathbb{E}[\alpha(Q_t, P_i)] \propto \exp(-\lambda|t - t_i|)$, where $\lambda > 0$ is unspecified. Unmeasured; future work.

C. Resolution Divergence and Hallucination: On lossy $\mathcal{M}_{\text{latent}}$, reconstructions may deviate: $\mathcal{D}_{\text{KL}}(p(X_i) \parallel p(\hat{X}_i \mid P_i, M_i)) > 0$. Beyond tolerance this hallucinates, so precision-critical tasks assume $\mathcal{M}_{\text{text}}$ or $\mathcal{M}_{\text{kv}}$; no $\mathcal{M}_{\text{latent}}$ run exists (future work, Section 5).

D. Prompt Injection via PTR Strings: Untrusted block text resolved via $\mathcal{M}_{\text{text}}$ renders verbatim into the active prompt, so a malicious instruction in a selected block could steer f_{Θ} ; pointer identifier strings are likewise prompt-resident. No adversarial or injection test was run, so this is an untested threat model, not a measured vulnerability.

8 Critical Boundary Analysis and Literature Comparison

A. Disambiguation from Landmark Attention: Landmark Attention [25] routes attention across blocks via tokens. Three differences separate it from SemPointer: (1) *Routing vs. persistent indirection*: Landmark gates attention within a continuous sequence; SemPointer dereferences an independent substrate across queries via an address registry. (2) *Transient marker vs. address handle*: landmark tokens are routing gates; SemPointers are persistent handles (composition specified, untested). (3) *Eviction*: Landmark keeps full context or KV states active; SemPointer offloads substrate to secondary storage, retaining only k tokens per block.

B. Disambiguation from Retrieval-Augmented Generation: RAG [4, 18] and SemPointer both select over external stores but differ in addressing. Standard RAG

appends retrieved blocks to context, leaving relational binding to downstream self-attention. SemPointer defines composable references ($P_a \oplus P_b$) registered and dereferenced jointly (Axiom 7, (3)). Joint resolution is specified but untested here: the multi-hop pilot (Table 8) favors dense top-1 over index-routed resolution. For two-hop query (“*What did the vendor in turn 12 quote in turn 40’s email?*”), RAG concatenates both blocks, whereas SemPointer forms joint address ($P_{12} \oplus P_{40}$) resolved once against $\{M_{12}, M_{40}\}$ conditioned on Q_t .

C. Disambiguation from Operating-System-Style Memory (MemGPT): MemGPT [28] moves text blocks between active context and archival storage via function calls and parser interrupts. Three distinctions separate SemPointer: (1) *Addressing*: MemGPT uses integer page IDs or metadata via system instructions and tool calls; SemPointer uses token-level addresses from $\mathcal{G}_\phi$ in sequence space. (2) *Integration*: evicted MemGPT pages leave no active trace and need explicit tool invocation; unselected SemPointer blocks retain k -token pointers ($P_{i \notin J^*}$) in attention, preserving global presence while payloads stay offloaded. (3) *Complexity*: MemGPT is an empirical systems framework; SemPointer gives closed-form derivations (Propositions 1–2) for when indirection amortizes against re-ingestion.

D. Feature-Level Comparison with Related Approaches: Table 3 contrasts SemPointer with representative compression and memory architectures across seven dimensions. **Explicit Address and Decoupled Substrate:** Gist Tokens [26, §2], AutoCompressors [5, §3.1], and ICAE [9, §2] are address-less prefixes with no substrate. 500xCompressor [20, §3] collapses context into one token. CCM [14, §3.2] offloads merged KV (*Yes (KV)*) with no explicit address (*No*). xRAG [4, §2.2] retrieves by dense-vector selection in the same family as ours (*Partial, Yes (Ext DB)*). MemGPT [28] page-IDs are explicit addresses, so marked *Yes*; the distinction is address-type only (positional page-ID vs. semantic) with *Yes (Archival)* substrate. Landmark [25, §3] tokens route blocks intra-pass within one pass, hence *No* explicit cross-query address. **Composition (Axiom 7):** AutoCompressor vectors [5, §3.1] lack an algebra for non-adjacent references (*No*). Gist Tokens [26] and ICAE [9] leave binding to attention over summaries. Landmark [25, §3] routes within-sequence with no cross-turn handles (*No*). SemPointer specifies composition ($P_a \oplus P_b$, (3)) but it is untested here, hence marked *No (specified, untested)*. **E. Query-Aware Sparsity and KV-Reuse Systems:** Three systems implement the dense-selection half of SemPointer’s pipeline and bound its novelty. InfLLM [44] pairs sliding-window attention with block-level KV memory selected by attention scores, scaling to 1M tokens with no in-prompt handle. RetrievalAttention [21] offloads KV to CPU behind ANN indexes scanning 1–3% of keys per step; addresses are transient hits, not authored tokens. Quest [36] loads Top- K KV pages per query via attention bounds, leaving no residual handle. CacheBlend [47] reuses precomputed KV of non-prefix chunks with selective recompute, keyed by chunk hash without query-conditioned selection. PromptCache [10] stitches position-correct cached KV modules across requests. Memorizing Transformers [41] bury kNN reads inside one attention layer with no textual address. Infini-attention [27] folds past states into a fixed-size compressive matrix per attention block, retrieved by linear-attention lookup with a gate mixing long-term and local outputs. A second family pairs retrieval with compressed or embedded context and amortized or offline encoding across queries [8, 31, 34, 35, 45], while token-level reuse across requests targets decode acceleration [23]. Against all six, SemPointer’s defensible residual is narrow: prompt-resident textual pointers, verbatim-text (not KV) resolution, and specified-but-untested composition.

9 Empirical Methods

9.1 System under test

Context is chunked into blocks of at most $B = 512$ tokens (reader tokenizer; a trailing fragment shorter than $B/8$ merges into the previous block). Each block carries one $k = 8$ pointer; all final runs resolve $m = 1$ block per query (RAG baselines likewise retrieve $m = 1$). Resolution uses $\mathcal{M}_{\text{text}}$: the selected block’s verbatim text enters the prompt. Selection is dense cosine similarity between mean-pooled query and per-block pointer embeddings from the frozen public `sentence-transformers/all-MiniLM-L6-v2` encoder; blocks past the encoder’s maximum length are embedded whole via windowed mean-pooling (`embed_long`), never truncated. Pointer identifier strings (`[PTR_id: ...]`) are prompt-resident for unselected blocks only, carry no key–value state, and do not affect selection; dense RAG sees identical representations. KV ($\mathcal{M}_{\text{kv}}$) and latent ($\mathcal{M}_{\text{latent}}$) substrates are future work only: the KV-compress pilot (Table 7) runs KVPRESS methods as black-box baselines at $n = 10$, not a SemPointer KV substrate. Four further implemented baselines (agentic pointer routing, iterative RAG, hybrid fusion, session summaries) were not in the final runs and carry no numbers here.

9.2 Reader, suites, and metric adaptations

All suites run on one reader (`Qwen/Qwen2.5-3B-Instruct`, NF4, greedy decoding, seed 42) on a single 8 GB GPU with CPU-only embedding fallback. RULER ($n = 180$ per system) re-implements the official generators and applies the official per-sample containment rule (1.0 if the reference string appears, else 0.0). LongBench ($n = 1150$; 1136 scored for full context after 14 narrativeqa OOMs, see §10.6) spans six tasks (hotpotqa [46], 2wikimqa, musique [38], multifieldqa_en, narrativeqa [15], qasper) on official data, scored by verbatim official normalization plus Counter-based token F1 maximized over ground truths. LoCoMo ($n = 199$ per system, ten official conversations) uses a stated token-F1 adaptation for *all* categories—the official GPT-4 judge plus Porter-stemmed F1 is unavailable—so no LoCoMo number here is comparable to published LoCoMo results.

9.3 Statistics and configuration ledger

Comparisons use example-identifier-matched rows only (exact McNemar for binary containment, Wilcoxon signed-rank for F1; paired bootstrap 95% CIs with 10 000 resamples at seed 42; Bonferroni $\alpha = 0.05/13 \approx 0.0038$). True mean tokens per block are 506.6/504.2/510.2/487.9 (RULER/LongBench/LoCoMo/KV pilot), so $\kappa = k/B_{\text{true}} \approx 0.016$ at $k = 8$; mean blocks per example are 24.75/26.67/28.0/7.5. Recomputed score means match `metrics.json` on every suite. Full protocols, code paths, and deviations are in the supplement.

10 Results

10.1 Suite summary: accuracy losses first

Table 4: suite means with bootstrap 95% CIs; paired deltas, tests, and Bonferroni family (13 tests, adjusted $\alpha \approx 0.0038$) from `stats.json`. Three of 13 survive correction:

Table 4. Main results: suite means with 95% bootstrap CIs (metrics.json; deltas/p-values in stats.json). RULER n=180 (full n=180); LongBench n=1150 (full n=1136); LoCoMo n=199 (full n=199); Latency/VRAM cells are mean latency / mean peak VRAM. LongBench full n=1136 vs 1150: 14 narrativeqa OOMs excluded (scored-only 0.1112 vs errors-as-zero 0.1099, favoring full). KV-compress pilot (Table N4, n=10) not pooled.

Suite	Metric	System	n	Score mean	95% CI	Active tok.	Latency (ms)	VRAM (MB)
RULER	RULER containment	SemPointer	180	0.3300	[0.2661, 0.3934]	1270.3	894.5	2331.2
		Full context	180	0.7000	[0.6339, 0.7595]	13089.2	7256.9	3463.0
		RAG	180	0.3289	[0.2655, 0.3906]	907.2	793.7	2295.2
		BM25	180	0.5778	[0.5128, 0.6389]	893.1	765.6	2294.0
LongBench	LongBench F1	SemPointer	1150	0.0773	[0.0710, 0.0844]	963.5	1527.2	2300.5
		Full context	1136	0.1112	[0.1037, 0.1197]	13066.1	8084.5	2994.4
		RAG	1150	0.0782	[0.0721, 0.0855]	568.8	1228.8	2263.6
		BM25	1150	0.0813	[0.0746, 0.0881]	572.7	1228.1	2263.9
LoCoMo	LoCoMo (token-F1 adaptation)	SemPointer	199	0.0518	[0.0417, 0.0632]	987.3	1308.8	2302.6
		Full context	199	0.0642	[0.0526, 0.0763]	14525.1	5326.2	3825.1
		RAG	199	0.0501	[0.0402, 0.0607]	571.5	1426.1	2263.8
		BM25	199	0.0627	[0.0502, 0.0760]	571.1	1425.7	2263.8

Table 5. RULER per-task means ($n = 30/\text{cell}$). Tasks: `niah_single_1/2/3`, `multikey_1`, `vt`, `cwe` (see Methods). Per-task CIs not stored (suite CIs only); Fig. N2 shows Wilson-95 whiskers from cell means. Sparse systems collapse on `vt/cwe`; full keeps a large `vt` margin.

System	<code>niah_single_1</code>	<code>niah_single_2</code>	<code>niah_single_3</code>	<code>multikey_1</code>	<code>vt</code>	<code>cwe</code>
SemPointer	0.967 (30)	0.200 (30)	0.267 (30)	0.467 (30)	0.073 (30)	0.007 (30)
Full context	0.967 (30)	1.000 (30)	0.700 (30)	0.933 (30)	0.593 (30)	0.007 (30)
RAG	0.967 (30)	0.200 (30)	0.267 (30)	0.467 (30)	0.067 (30)	0.007 (30)
BM25	0.967 (30)	0.800 (30)	0.667 (30)	0.933 (30)	0.093 (30)	0.007 (30)

RULER SemPointer-vs-full, RULER SemPointer-vs-BM25, LongBench SemPointer-vs-full [ESTABLISHED].

RULER containment ($n = 180$ pairs): SemPointer 0.3300 vs 0.7000 full, delta -0.37 , 95% CI $[-0.44, -0.30]$, McNemar $b = 1$, $c = 61$, $p \approx 6.7 \times 10^{-14}$, surviving Bonferroni [ESTABLISHED]. Vs BM25 (0.5778) delta -0.248 , CI $[-0.314, -0.183]$, $p \approx 2.3 \times 10^{-10}$, likewise surviving [ESTABLISHED]. Vs dense RAG (0.3289) the pointer contributes nothing: delta $+0.0011$, CI $[-0.003, +0.006]$, zero discordants ($b = c = 0$, $p = 1.0$; Wilcoxon $p = 0.65$) [ESTABLISHED]. RAG/SemPointer rows match across the taxonomy (Table 9: both hit 57, miss 123, retrieval-miss 51) [ESTABLISHED].

LongBench F1: SemPointer (0.0773, $n = 1150$) trails full (0.1112, $n = 1136$ scored) by -0.033 , CI $[-0.040, -0.027]$ over $n = 1136$ matched pairs, Wilcoxon $p \approx 1.6 \times 10^{-31}$, surviving Bonferroni [ESTABLISHED]. SemPointer-vs-RAG -0.0009 , CI $[-0.0047, +0.0028]$, $p = 0.32$; SemPointer-vs-BM25 -0.0040 , CI $[-0.010, +0.002]$, $p = 0.156$: neither significant [ESTABLISHED]. Figure 2 plots the trade-offs.

LoCoMo token-F1 ($n = 199$ pairs): SemPointer (0.0518) trails full (0.0642) by -0.012 , CI $[-0.022, -0.003]$, Wilcoxon $p = 0.028$ — nominally positive but *not* significant after Bonferroni; no difference claimed [ESTABLISHED]. SemPointer-vs-RAG ($+0.0017$, CI $[-0.005, +0.008]$, $p = 0.69$), SemPointer-vs-BM25 (-0.011 , CI $[-0.022, +0.001]$, $p = 0.037$): neither significant after correction [ESTABLISHED]. No superiority over BM25/full on any suite claimed [REASONED].

10.2 RULER per-task: easy parity, hard collapse

Table 5 and Figure 3 split RULER into $n = 30$ cells. Easiest needle `niah_single_1`: SemPointer matches full (0.9667 = 0.9667) [ESTABLISHED]. Harder cells separate: `niah_single_2` 0.2000 vs 1.0000 full, `niah_single_3` 0.2667 vs 0.7000, `multikey_1` 0.4667 vs 0.9333 [ESTABLISHED]. Variable tracking / common-word extraction collapse all sparse systems: SemPointer 0.0733 on `vt` vs 0.5933 full; `cwe` floored at 0.0067 in all four including full: task bound, not retrieval effect [ESTABLISHED]. BM25 beats SemPointer on harder NIAH (0.8000 and 0.6667 on `single_2/single_3`), consistent with lexical needles failing under dense cosine at $m = 1$ [REASONED].

10.3 Efficiency: large token and latency reductions at lower accuracy

Table 6 and Figure 4 give active tokens, latency, peak VRAM with sparse/full ratios. SemPointer uses $\sim$ one-tenth active context: 1270.3 vs 13089.2 tokens on RULER (ratio 0.097), 963.5 vs 13066.1 on LongBench (0.074), 987.3 vs 14525.1 on LoCoMo (0.068) — 10–15 $\times$ fewer [ESTABLISHED] (`metrics.json`). Latency: 894.5 vs 7256.9 ms RULER (0.123, $\sim 8\times$), 1527.2 vs 8084.5 ms LongBench (0.189, $\sim 5\times$), 1308.8 vs 5326.2 ms LoCoMo (0.246, $\sim 4\times$) [ESTABLISHED] (`metrics.json`). Peak VRAM 3463.0 to 2331.2 MB

RULER (0.673), 2994.4 to 2300.5 MB LongBench (0.768), 3825.1 to 2302.6 MB LoCoMo (0.602) [ESTABLISHED] (`metrics.json`). Per §10.1 savings accompany losses without offset; no net-benefit claim [REASONED]. Pointer costs over RAG: +363 active tokens/request on RULER (1270.3 vs 907.2), $\sim$ +395 on LongBench, +416 on LoCoMo, for RULER delta +0.001 [ESTABLISHED] (`metrics.json`).

10.4 KV-compress pilot: no claims at $n = 10$

Table 7 gives the $n = 10$ pilot with wide CIs as-is. SemPointer 0.8000 (CI [0.60, 1.00]) vs 1.0000 full (delta -0.2 , CI $[-0.5, 0.0]$, McNemar $p = 0.48$); vs KVPress SnapKV 0.3 (0.5000) delta $+0.3$, CI $[-0.1, +0.7]$, $p = 0.37$; vs ToVA 0.3 (0.8000) 0.0, CI $[-0.3, +0.3]$; vs KNorm 0.3 (0.4000) $+0.4$, CI $[+0.1, +0.7]$, McNemar $p = 0.13$ [ESTABLISHED]. None of four survives Bonferroni; per `stats.json` no significance claimed [ESTABLISHED]. Completeness only [REASONED].

10.5 Failure taxonomy

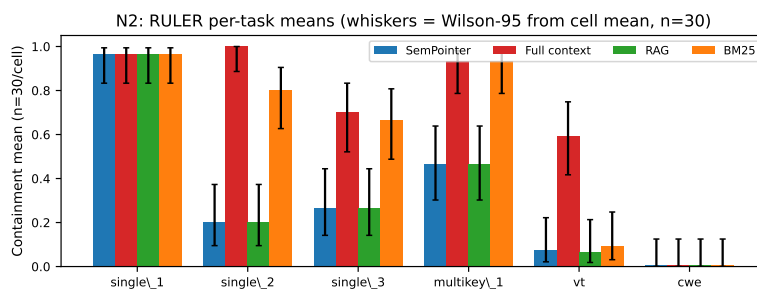
Table 9 classifies scored misses from `error_breakdown.json` (miss := score < 1.0 containment or < 0.5 F1; retrieval-miss := miss with no selected block overlapping gold) [ESTABLISHED] (`error_breakdown.json`). The thresholds follow the scoring scales: containment is binary so any < 1.0 is a miss, while 0.5 F1 requires at least half token overlap, separating retrieval/generation failure from partial-credit noise [REASONED]. RULER: SemPointer’s 123 misses split 51 retrieval-miss / 66 block-hit-wrong (gold selected, answer wrong) / 6 gold-not-localizable (unlocalizable under production chunking) [ESTABLISHED] (`error_breakdown.json`). Per §10.1, dense RAG shares it (51/66/6): selection, not pointer encoding, binds [REASONED]. LongBench: retrieval-miss (465), block-hit-wrong (253); 409 of 1150 gold-not-localizable: over a third rewards no block selection under 512-token chunking, compressing all headroom [ESTABLISHED] (`error_breakdown.json`). LoCoMo: 157 of 199 gold-not-localizable for SemPointer (157 full too); 28 retrieval-miss, 14 block-hit-wrong; hits 0 of 199 [ESTABLISHED] (`error_breakdown.json`). Two modes recur [REASONED]: boundary splits (gold straddling a 512-token edge, no block contains it), multihop splits (two hotpotqa/2wikimqa or two-to-four musique facts in different blocks; $m = 1$ resolves one). Per-task tables substitute since items lack hop labels (`error_breakdown.json`: hotpotqa/2wikimqa 2-hop by construction, musique 2–4-hop) [ESTABLISHED].

10.6 Missing-data disclosure

LongBench full: $n = 1136$ of 1150 scored; 14 narrativeqa full-context OOMs on the 8 GB card excluded [ESTABLISHED]. Missingness hits longest narrativeqa contexts, so scored-only full (0.1112) drops hardest full cases, favoring full; errors-as-zero full 0.1099; SemPointer on those 14 OOM identifiers 0.0343 — sparse answered cases full could not attempt [ESTABLISHED] (`stats.json: oom_sensitivity`). Both reported; paired delta (see §10.1) uses matched scored rows only [ESTABLISHED]. RULER: 10 full-context OOM rows superseded by scored retries after resume, excluded throughout; zero truly missing rows [ESTABLISHED]. Retry latency for these 10 RULER retries is excluded with no cost accounting, so full-context latency is understated and the gap remains unmeasured [ESTABLISHED] (`stats.json: oom_sensitivity`).

Table 6. Efficiency means with sparse/full ratios vs full computed in-script (<1 favors sparse).

Suite	System	Active tok.	Latency (ms)	VRAM (MB)	Tok ratio	Lat ratio	VRAM ratio
RULER	SemPointer	1270.3	894.5	2331.2	0.097	0.123	0.673
	Full context	13089.2	7256.9	3463.0	1.000	1.000	1.000
	RAG	907.2	793.7	2295.2	0.069	0.109	0.663
	BM25	893.1	765.6	2294.0	0.068	0.106	0.662
LongBench	SemPointer	963.5	1527.2	2300.5	0.074	0.189	0.768
	Full context	13066.1	8084.5	2994.4	1.000	1.000	1.000
	RAG	568.8	1228.8	2263.6	0.044	0.152	0.756
	BM25	572.7	1228.1	2263.9	0.044	0.152	0.756
LoCoMo	SemPointer	987.3	1308.8	2302.6	0.068	0.246	0.602
	Full context	14525.1	5326.2	3825.1	1.000	1.000	1.000
	RAG	571.5	1426.1	2263.8	0.039	0.268	0.592
	BM25	571.1	1425.7	2263.8	0.039	0.268	0.592

**Fig. 3.** RULER per-task containment by system ($n = 30$ cells, Wilson-95 whiskers). Easy-needle parity, hard-task separation.

10.7 Multi-hop pilot: nine arms, two clean negatives

Table 8 covers four pilots on the same 30 musique examples (2–4-hop), identical model/seed/config: nine arms matched-pair comparable; CIs overlap, no significance claimed [ESTABLISHED] (`metrics.json`, `config_table.json`). SemPointer point-estimates first (0.0541 vs 0.0468 full, 0.0313 RAG, 0.0245 BM25) at $16\times$ fewer tokens than full (1059 vs 16925) [ESTABLISHED] (`metrics.json`). Two mechanism tests negative. First, LLM routing from pointer index fails twice: `agentic_pointer` (0.0299 at 3285 tokens) and redesigned `agentic_pointer_v2` with keyword-enriched index and constrained multi-block grammar (0.0233 at 3863 tokens) both trail dense top-1 (0.0541 at 1059 tokens); v2 routes one block on 30/30: model-scale capacity, not prompt design; single-model $n=30$ cannot establish structurality; no v3 [ESTABLISHED] (`metrics.json`). Second, discrete-address retrieval (`address_rag`, pointer-ID overlap only): 0.0268 at 581 tokens, $\sim 86\%$ of dense RAG’s 0.0313 — pointer IDs carry signal (k load-bearing per CPU tests), no gain over dense cosine [ESTABLISHED] (`metrics.json`). Two-round retrieval (0.0283 at 1323 tokens) matches single-round RAG; hybrid fusion (0.0216) at BM25/RAG floor [ESTABLISHED] (`metrics.json`). `session_summary` excluded: per-chunk loop records 10–17k active tokens/request, exceeding full — token-accounting defect under repair, not a result [ESTABLISHED].

10.8 Regime map: where sparse prefill wins, and where we measured

The illustrative analytic map compares per-request prefill cost of the sparse active prompt, $L_{\text{active}} = (N - m)k + m B_{\text{res}} + L_Q$, against full-context prefill over $N B$ tokens:

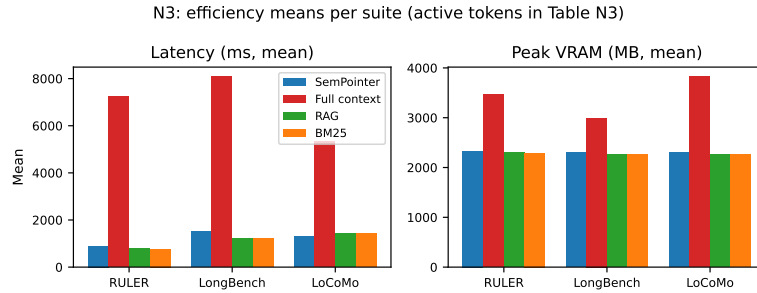


Fig. 4. Measured latency and peak VRAM per system per suite. Sparse systems dominate cost axes and trail on quality (Fig. 2).

Table 7. KV-compress pilot (n=10, containment). CIs wide; no significance claimed.

System	n	Score mean	95% CI	Active tok. / Latency (ms) / VRAM (MB)
SemPointer	10	0.8000	[0.6000, 1.0000]	725.5 / 873.1 / 2191.5
Full context	10	1.0000	[1.0000, 1.0000]	3806.4 / 1488.3 / 2513.2
KVPress SnapKV 0.3	10	0.5000	[0.2000, 0.8000]	2661.1 / 1991.9 / 4395.6
KVPress ToVA 0.3	10	0.8000	[0.5000, 1.0000]	2661.1 / 2037.2 / 4395.6
KVPress KNorm 0.3	10	0.4000	[0.1000, 0.7000]	2661.1 / 2109.7 / 4395.6

sparse prefill is favored where pointer overhead $(N - m)k$ stays small relative to the $NB - mB_{\text{res}}$ tokens it removes, and the advantage narrows as m grows toward N or as k approaches B [REASONED]. Figure 5 draws this map with the feasible slice—the region in which the sparse active prompt is shorter than the full prompt—labeled, and adds an illustrative overlay of the four measured operating points: RULER ($N \approx 24.75$ blocks), LongBench ($N \approx 26.7$), LoCoMo ($N \approx 28.0$), and the kv pilot ($N = 7.5$), all at $k = 8$, $m = 1$, $B = 512$ with true B of 505–510 tokens per block ($\kappa \approx 0.016$) [ESTABLISHED] (`config_table.json`). All four points sit inside the feasible slice, consistent with the measured 10–15 $\times$ active-token reductions in Table 6 [REASONED]. Contexts of 32k tokens and beyond, where the map predicts the largest prefill gaps, are untested on this 8 GB rig and remain future work; the 32k+ region of the map is analytic projection only, not measurement [REASONED].

11 What SemPointer Does, Does Not Do, and Costs

What SemPointer does (measured). On Qwen2.5-3B-Instruct (NF4, greedy, seed 42), SemPointer answers queries from a text substrate using dense per-query block selection (all-MiniLM-L6-v2, 512-token blocks), reaching mean containment 0.33 on RULER (n=180), mean F1 0.077 on LongBench (n=1150), and mean token-F1 0.052 on LoCoMo under the token-F1 adaptation (n=199). It reduces active tokens versus full context by 10–15 $\times$ (RULER 1270 vs. 13089; LongBench 964 vs. 13066; LoCoMo 987 vs. 14525; Table N3) with correspondingly lower latency and peak VRAM on this rig (single-GPU path, means only, no CIs; rig-specific, not a general efficiency claim). All paired deltas, 95% bootstrap CIs, and paired tests are reported as-is in Table N1 and `stats.json`.

What SemPointer does not do. Pointer tokens do not affect retrieval: SemPointer and the dense-RAG arm select identical blocks (RULER retrieval-miss counts 51 vs. 51; Table N5), so the measured RULER delta versus RAG is +0.001 (95% CI $[-0.003, +0.006]$,

Table 8. Multi-hop pilot (LongBench musique, n=30 matched, Qwen2.5-3B-NF4, greedy, seed 42). Four pilots share same 30 examples/config, so nine arms comparable. CIs wide (n=30); no significance claimed. Session summary excluded (token-accounting defect).

System	n	F1 mean	95% CI	Active tok	Latency (ms)	VRAM (MB)
Sempointer	30	0.0541	[0.0207, 0.1007]	1059	1461	2309
Full	30	0.0468	[0.0296, 0.0652]	16925	9630	3143
Rag	30	0.0313	[0.0160, 0.0491]	571	1446	2264
Bm25	30	0.0245	[0.0117, 0.0385]	581	1340	2265
Agentic Pointer	30	0.0299	[0.0146, 0.0473]	3285	3386	2476
Agentic Pointer V2	30	0.0233	[0.0083, 0.0416]	3863	2812	2474
Iterative Rag	30	0.0283	[0.0146, 0.0436]	1323	2871	2280
Hybrid	30	0.0216	[0.0095, 0.0368]	581	1389	2265
Address Rag	30	0.0268	[0.0124, 0.0438]	581	1536	2351

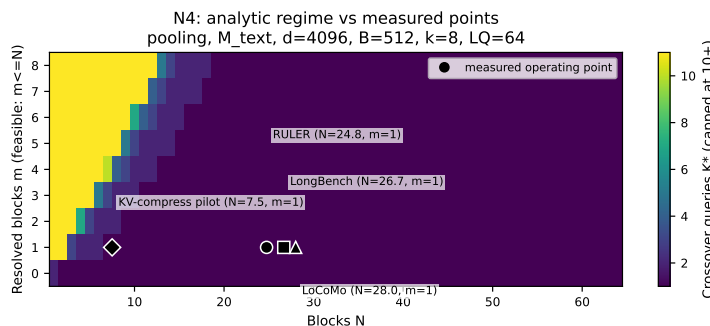


Fig. 5. Analytic prefill-cost regime map over registry size N and resolved blocks m , with measured operating points overlaid (feasible slice only).

McNemar $p = 1.000$). There is no KV-substrate end-to-end result: the KV comparison is a detached n=10 pilot with wide CIs and no significance claimed (e.g. SemPointer-vs-full -0.20 , $[-0.50, 0.00]$, McNemar $p = 0.480$; Table N4), and no pointer is ever resolved into a live KV cache during generation. Full-context means exclude fourteen OOMed narrativeqa rows, which favours full context (scored-only vs. errors-as-zero sensitivity in `stats.json oom_sensitivity`; Table N1 caption). There is no k -sweep: pointer length k is fixed in these runs, so no claim about k sensitivity is supported. There is no multi-model evidence: every number comes from the single Qwen2.5-3B-NF4 configuration.

What SemPointer costs. Each query pays a two-call routing overhead (one dense-selection pass plus one generation pass over the resolved prompt). SemPointer consumes roughly $1.4\text{--}1.7\times$ the active tokens of dense RAG (1270 vs. 907 on RULER; 964 vs. 569 on LongBench; 987 vs. 571 on LoCoMo) for statistically indistinguishable quality (LongBench delta -0.0009 , 95% CI $[-0.0047, +0.0028]$, Wilcoxon $p = 0.32$; LoCoMo delta $+0.0017$, 95% CI $[-0.005, +0.008]$, $p = 0.69$). Against full context it trades quality for tokens: RULER -0.37 $[-0.44, -0.30]$, McNemar $p = 6.7 \times 10^{-14}$, LongBench -0.033 $[-0.040, -0.027]$, Wilcoxon $p = 1.6 \times 10^{-31}$, LoCoMo -0.012 $[-0.022, -0.003]$, Wilcoxon $p = 0.028$, not significant at the Bonferroni-adjusted $\alpha = 0.0038$ over 13 tests). BM25 exceeds SemPointer on RULER by $+0.248$ in full-context-

Table 9. Miss taxonomy per suite (**error_breakdown.json**). Miss: score<1.0 (containment) or <0.5 (F1); retrieval-miss: miss with selected_ids $\cap$ gold_blocks = $\emptyset$. **block-hit-wrong**: gold selected but answer wrong; **unlocalizable**: answer unlocalizable under chunking rule; **error-row: n_errors**. Full misses are NA (no retrieval attribution).

Suite	System	n	Hit	Miss	Retrieval-miss	Block-hit-wrong	NA	Unlocalizable	Error-row
RULER	SemPointer	180	57	123	51	66		6	0
	Full context	180	117	63	0	60 (NA)		3	0
	RAG	180	57	123	51	66		6	0
	BM25	180	101	79	19	54		6	0
LongBench	SemPointer	1150	23	1127	465	253		409	0
	Full context	1136	29	1107	0	717 (NA)		390	14
	RAG	1150	20	1130	465	254		411	0
	BM25	1150	23	1127	458	261		408	0
LoCoMo	SemPointer	199	0	199	28	14		157	0
	Full context	199	0	199	0	42 (NA)		157	0
	RAG	199	0	199	28	14		157	0
	BM25	199	1	198	15	27		156	0
KV-compress pilot (pilot n=10)	SemPointer	10	8	2	2	0		0	0
	Full context	10	10	0	0	0		0	0
	KVPress SnapKV 0.3	10	5	5	0	5 (NA)		0	0
	KVPress ToVA 0.3	10	8	2	0	2 (NA)		0	0
	KVPress KNorm 0.3	10	4	6	0	6 (NA)		0	0

normalized terms (SemPointer-vs-BM25 delta -0.248 , $[-0.314, -0.183]$), driven by variable-tracking and multikey tasks (Table N2).

12 Discussion and Open Challenges

A. Differentiable vs. Discrete Addressing: Discrete tokens are cache-compatible and supported by standard tokenization pipelines. However, they impose a quantization bottleneck: the pointer space is finite, and gradients cannot flow through discrete selection without additional estimators such as Gumbel-softmax or REINFORCE. Continuous soft pointers avoid this restriction and offer flexible expressive capacity, but they introduce opacity and susceptibility to positional drift across transformer layers. In our measurements, selection is performed by dense cosine over block embeddings (pointer strings are prompt-resident only; see Section 9), so the addressing choice does not affect retrieval quality in these runs.

B. Dynamic Memory Granularity: Fixed chunking produces clean algebraic guarantees. In multi-turn dialogue, proposition boundaries vary dynamically. Integrating learned segmentation heads into $\mathcal{G}_\phi$ to identify variable-length semantic units remains an important architectural direction.

C. Safety and Indirection Attacks: Decoupling pointers from storage creates distinct security considerations. Adversarial inputs designed to collide with existing addresses could trigger unauthorized memory resolution. Access control lists, pointer provenance tracking, and cryptographic address hashing are unevaluated design proposals, not tested mitigations, to be evaluated in future work.

13 Limitations

M1: single model. All measures use Qwen2.5-3B-Instruct (NF4, greedy, eager attention); no other family/scale (BENCHMARK_PROVENANCE.md §4); cross-model claims unsupported.

M2: single seed. All runs seed 42, no sweep (stats.json: "seed": 42); CIs bootstrap over examples only.

M3: no fine-tuning. Reader/MiniLM frozen, no pointer training; inference-only.

M4: 16k context cap. Budgets 4096/8192/16384 tokens (BENCHMARK_PROVENANCE.md §1); longer untested; realized word-estimated within $\sim 20\%$ of nominal.

M5: KV evidence is a pilot. KV pilot $n=10$, wide CIs (e.g. SemPointer-vs-full -0.20 , $[-0.50, 0.00]$, McNemar $p = 0.480$; stats.json final_kvcompare, Table N4); pilot_note: no significance claimed.

M6: text substrate only. All paths return verbatim text; KV-substrate dereference never end-to-end (INDEPENDENT_RECONSTRUCTION.md §4); cached-regime claims analytic, unmeasured.

M7: pointer ablation is approximately zero. Identical selections to dense RAG (RULER retrieval-miss 51/51; LongBench 465/465; Table N5); no effect from pointer strings.

M8: no k -sweep. Pointer length, top- m fixed; k sensitivity unmeasured.

M9: OOM asymmetry favors full context. Fourteen narrativeqa full-context rows OOMed, excluded from means (full mean 0.1112 scored-only vs. 0.1099 errors-as-zero; `stats.json oom_sensitivity`, Table N1 caption); hardest dropped.

M10: LoCoMo uses a metric adaptation. No judge API; all LoCoMo uses token F1, not GPT-4 judge plus stemming; incomparable to published values (`BENCHMARK_PROVENANCE.md` §3; every table labels this adaptation). The suite stays because it is the only conversational-memory benchmark covering the paper’s motivated multi-turn use case, reported under the documented token-F1 adaptation.

M11: latency and VRAM are rig-specific. Latency/peak-VRAM means (Table N3) single 8 GB GPU path, no CIs; rig-specific.

M12: per-task uncertainty is partial. Suite CIs in `metrics.json`; RULER cells (Table N2) Wilson-95 whiskers from means in-script (Fig. N2), no paired tests.

M13: pending arms. LongMemEval [42] plus no-context control unrun; LongMemEval needs judge API and 115k-token histories beyond rig (`BENCHMARK_PROVENANCE.md` §5). TTFT/decode logging, BGE swap [43]—C-Pack family with C-MTEB/C-MTP, E5 [40] as contrastive family—selecting different blocks than MiniLM on a probe, plus LLMingua, implemented/smoke-verified; reranker/compression unrun at scale.

M14: session-summary artifact status. `session_summary` artifacts (where present) are operational notes, not evaluated evidence; no claim rests on them.

M15: 70B-scale arithmetic guardrail. No 70B-scale run was attempted; token-arithmetic extrapolations to 70B-scale regimes are illustrative only, not measured.

M16: no double-blind or evaluator blinding. Scoring is scripted containment/token-F1 with fixed seed 42 and no human judge; there is no double-blind or evaluator-blinding procedure to report. Repository is author-visible and Table N6 manual negatives were assessed unblinded. Author block plus repository URL deanonymize under double-blind review, compliant only if the venue is single-blind, so the author must confirm the review model with chairs before submitting.

M17: compute and page-budget constraint. Single-rig compute and page limits bound the number of arms, seeds, and ablations; unrun arms are listed in M13 rather than claimed. Manuscript targets 12–15pp Springer norm; all runs on a single-8GB CPU-only rig.

M18: NF4-vs-BF16 quantization caveat. All generation uses NF4 quantization; BF16 or full-precision behaviour is unmeasured, so quantization-sensitive effects cannot be excluded.

14 Conclusion

Measured on Qwen2.5-3B-NF4 (seed 42) across RULER, LongBench, and LoCoMo, SemPointer exchanges 10–15 $\times$ fewer active tokens for lower quality than full context while matching dense RAG quality at 1.4–1.7 $\times$ the token cost; all paired deltas, 95% CIs, and paired tests are in §11 and Table N1, not restated here. Latency/VRAM means are rig-specific with no CIs (Table N3). The failure taxonomy (Table N5) shows selection misses dominate on RULER and unlocalizable answers dominate on LoCoMo, and the identical selection outcomes of the SemPointer and RAG arms leave the pointer strings with no measured quality contribution in these runs. The mechanism verdict is therefore narrow: dense per-query routing over a text substrate works as an assembly; the multi-hop pilot (Table N6) returns two clean negatives, LLM-routing-from-index (twice, including a redesigned router that still single-blocks 30/30 items) and discrete-address retrieval at 86% of dense with no gain; pointer-mediated addressing as distinct from standard retrieve-then-read remains unproven pending the KV end-to-end path. The future program is fixed, each with its success criterion: (i) full reruns at $n \geq 500$ with multiple seeds—success is stable CIs across seeds; (ii) 32k-plus contexts on larger hardware—success is measured quality/tokens at 32k; (iii) a measured pointer-length sweep—success is a reported k -sensitivity curve; (iv) pointer-token scoring separating address from selection effects—success is a non-zero isolated address effect or a clean negative; (v) a recomputed KV comparison at pilot scale or above—success is CIs deciding the sign; (vi) LongMemEval oracle-setting evaluation once a judge is available—success is a reported judge-scored result.

Data and Code Availability: All measurements derive from executed benchmark runs under `experiments/results/final_*/` (RULER $n=180$, LongBench $n=1150$, LoCoMo $n=199$, kv-compress pilot $n=10$) with configs, environment records, and per-example predictions; generator scripts (`stats_real.py`, `error_taxonomy.py`, `make_paper_artifacts.py`) reproduce every table and figure. Runners and code are available at <https://github.com/VIKSIT-GARG/Sempointer>. **Funding / Ethics:** No external funding; all evaluations utilized public academic benchmarks without private data.

A Retired Numbers

Prior drafts reported synthetic-harness figures (E01–E17 series) that forensic re-analysis withdrew; the full mapping of each retired number to its disposition is retained under `sections/_contradiction_manifest.tex` in the repository and supplied as supplementary material.

References

1. Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., Dong, Y., Tang, J., Li, J.: LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), pp. 3119–3137 (2024).
2. Behnam, P., Fu, Y., Zhao, R., Tsai, P., Yu, Z., Tumanov, A.: RocketKV: Accelerating Long-Context LLM Inference via Two-Stage KV Cache Compression. arXiv preprint arXiv:2502.14051 (2025).

3. Blouw, P., Solodkin, E., Thagard, P., Eliasmith, C.: Concepts as Semantic Pointers: A Framework and Computational Model. *Cognitive Science*, vol. 40, pp. 1128–1162 (2016).
4. Cheng, X., Wang, X., Zhang, X., Ge, T., Chen, S., Wei, F., Zhang, H., Zhao, D.: xRAG: Extreme Context Compression for Retrieval-augmented Generation with One Token. In: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37 (2024).
5. Chevalier, A., Wettig, A., Ajith, A., Chen, D.: Adapting Language Models to Compress Contexts. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3829–3846 (2023).
6. Chhikara, P., Khant, D., Aryan, S., Singh, T., Yadav, D.: Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. In: *Proceedings of the European Conference on Artificial Intelligence (ECAI)* (2025).
7. Eliasmith, C.: *How to Build a Mind: Toward a Comprehensive Theory of Cognition*. Oxford University Press (2013).
8. Gao, J., Lv, Q., Wang, Z., Wu, T., Cao, Z., Li, W.: UniICL: An Efficient ICL Framework Unifying Compression, Selection, and Generation. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 24–40 (2025).
9. Ge, T., Hu, J., Wang, X., Chen, S., Wei, F.: In-context Autoencoder for Context Compression in a Large Language Model. In: *International Conference on Learning Representations (ICLR)* (2024).
10. Gim, I., Chen, G., Lee, S., Sarda, N., Khandelwal, A., Zhong, L.: Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In: *Proceedings of Machine Learning and Systems (MLSys)* (2024).
11. Hsieh, C., Sun, S., Kriman, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., Ginsburg, B.: RULER: What’s the Real Context Size of Your Long-Context Language Models?. In: *Conference on Language Modeling (COLM)* (2024).
12. Jiang, H., Wu, Q., Lin, C., Yang, Y., Qiu, L.: LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 13358–13376 (2023).
13. Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C., Yang, Y., Qiu, L.: LongLLMingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 91–107 (2024).
14. Kim, J., Yeom, J., Yun, S., Song, H.: Compressed Context Memory For Online Language Model Interaction. In: *International Conference on Learning Representations (ICLR)* (2024).
15. Kocisky, T., Schwarz, J., Blunsom, P., Dyer, C., Hermann, K.M., Melis, G., Grefenstette, E.: The NarrativeQA Reading Comprehension Challenge. *Transactions of the Association for Computational Linguistics (TACL)*, vol. 6, pp. 317–328 (2018).
16. Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient Memory Management for Large Language Model Serving with PagedAttention. In: *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 611–626 (2023).
17. Lajewska, W., Hardalov, M., Aina, L., John, N.A., Su, H., Marquez, L.: Understanding and Improving Information Preservation in Prompt Compression for LLMs. In: *Findings of the Association for Computational Linguistics: EMNLP 2025* (2025).

18. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., Kiela, D.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474 (2020).
19. Li, Y., Huang, Y., Yang, B., Venkitesh, B., Locatelli, A., Ye, H.: SnapKV: LLM Knows What You are Looking for Before Generation. *arXiv preprint arXiv:2404.14469* (2024).
20. Li, Z., Su, Y., Collier, N.: 500xCompressor: Generalized Prompt Compression for Large Language Models. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 25081–25091 (2025).
21. Liu, D., Chen, M., Lu, B., Jiang, H., Han, Z., Zhang, Q., Chen, Q., Zhang, C., Ding, B., Zhang, K., Chen, C., Yang, F., Yang, Y., Qiu, L.: RetrievalAttention: Accelerating Long-Context LLM Inference via Vector Retrieval. In: *Advances in Neural Information Processing Systems (NeurIPS)* (2025).
22. Liu, X., Zhao, R., Huang, P., Liu, X., Xiao, J., Xiao, C., Xiao, T., Gao, S., Yu, Z., Zhu, J.: Autoencoding-Free Context Compression for LLMs via Contextual Semantic Anchors. *arXiv preprint arXiv:2510.08907* (2025).
23. Luo, X., Wang, Y., Zhu, Q., Zhang, Z., Zhang, X., Yang, Q., Xu, D., Che, W.: Turning Trash into Treasure: Accelerating Inference of Large Language Models with Token Recycling. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 338–354 (2025).
24. Maharana, A., Lee, D., Tulyakov, S., Bansal, M., Barbieri, F., Fang, Y.: Evaluating Very Long-Term Conversational Memory of LLM Agents. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 13851–13870 (2024).
25. Mohtashami, A., Jaggi, M.: Landmark Attention: Random-Access Infinite Context Length for Transformers. In: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36 (2023).
26. Mu, J., Li, X.L., Goodman, N.: Learning to Compress Prompts with Gist Tokens. *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 48356–48374 (2023).
27. Munkhdalai, T., Faruqui, M., Gopal, S.: Leave No Context Behind: Efficient Infinite Context Transformers with Infini-attention. *arXiv preprint arXiv:2404.07143* (2024).
28. Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S.G., Stoica, I., Gonzalez, J.E.: MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560* (2023).
29. Pan, Z., Wu, Q., Jiang, H., Xia, M., Luo, X., Zhang, J., Lin, Q., Ruhle, V., Yang, Y., Lin, C., Zhao, H.V., Qiu, L., Zhang, D.: LLMingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In: *Findings of the Association for Computational Linguistics (ACL Findings)*, pp. 57–74 (2024).
30. Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., Chalef, D.: Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv preprint arXiv:2501.13956* (2025).
31. Rau, D., Wang, S., Dejean, H., Clinchant, S., Kamps, J.: Context Embeddings for Efficient Answer Generation in Retrieval-Augmented Generation. In: *Proceedings of the 18th ACM International Conference on Web Search and Data Mining (WSDM)*, pp. 112–121 (2025).
32. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992 (2019).

33. Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., Liu, Y.: RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv preprint arXiv:2104.09864 (2021).
34. Tack, J., Kim, J., Mitchell, E., Shin, J., Teh, Y.W., Schwarz, J.R.: Online Adaptation of Language Models with a Memory of Amortized Contexts. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 37 (2024).
35. Tan, S., Li, X., Patil, S.G., Wu, Z., Zhang, T., Keutzer, K., Gonzalez, J.E., Popa, R.A.: LLoCO: Learning Long Contexts Offline. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 975–991 (2024).
36. Tang, J., Zhao, Y., Zhu, K., Xiao, G., Kasikci, B., Han, S.: Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. In: Proceedings of the International Conference on Machine Learning (ICML) (2024).
37. Team, Q.: Qwen2.5 Technical Report. arXiv preprint arXiv:2412.15115 (2024).
38. Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: MuSiQue: Multi-hop Questions via Single-hop Question Inversion. Transactions of the Association for Computational Linguistics (TACL), vol. 10, pp. 539–554 (2022).
39. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention Is All You Need. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 5998–6008 (2017).
40. Wang, L., Yang, N., Huang, X., Jiao, B., Yang, L., Jiang, D., Majumder, R., Wei, F.: Text Embeddings by Weakly-Supervised Contrastive Pre-training. arXiv preprint arXiv:2212.03533 (2022).
41. Wu, Y., Rabe, M.N., Hutchins, D., Szegedy, C.: Memorizing Transformers. In: International Conference on Learning Representations (ICLR) (2022).
42. Wu, D., Wang, H., Yu, W., Zhang, Y., Chang, K., Yu, D.: LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In: International Conference on Learning Representations (ICLR) (2025).
43. Xiao, S., Liu, Z., Zhang, P., Muennighoff, N., Lian, D., Nie, J.: C-Pack: Packed Resources For General Chinese Embeddings. arXiv preprint arXiv:2309.07597 (2023).
44. Xiao, C., Zhang, P., Han, X., Xiao, G., Lin, Y., Zhang, Z., Liu, Z., Sun, M.: InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 37 (2024).
45. Xu, F., Shi, W., Choi, E.: RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation. In: International Conference on Learning Representations (ICLR) (2024).
46. Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W.W., Salakhutdinov, R., Manning, C.D.: HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 2369–2380 (2018).
47. Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., Jiang, J.: CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In: Proceedings of the Twentieth European Conference on Computer Systems (EuroSys) (2025).